

# Comparison of Modelica and ModelingToolkit.jl

Madhusudhan Pandey

November 29, 2025

## Abstract

This note compares the modeling frameworks **Modelica** and **ModelingToolkit.jl (MTK)** across several dimensions relevant for physical-system modeling, highlighting strengths and trade-offs for research and engineering contexts.

## Comparison Table

| Aspect                                | Modelica                                                                                                                                                                                                                            | ModelingToolkit.jl (MTK)                                                                                                                                                                                                                                                                                             |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Connectors / Ports                    | Built-in port/connector mechanism; supports across (potential) and through (flow) variables. Connection automatically enforces potential equality and flow conservation (plus mass/energy/stream balance in fluid/thermal domains). | Connector semantics via symbolic variables. Components use user-defined connectors (e.g. via <code>@connector</code> ); composition yields a global symbolic system which MTK simplifies and lowers. Connector types can be custom-defined (flow, stream, domain-specific).                                          |
| Multi-domain & Libraries              | Mature, broad standard libraries covering mechanical, electrical, fluid/thermal, control, multi-domain coupling; many ready-to-use components.                                                                                      | Ecosystem is younger but growing; as MTK is open-source and composable in Julia, users can build custom domain libraries (e.g. fluid + mechanical + control + thermal) and compose them flexibly.                                                                                                                    |
| PDE / Advanced Modeling               | PDE or spatial modeling possible via discretization or third-party libraries — depends strongly on library/tool; support not uniform across all domains.                                                                            | High flexibility: symbolic definitions, discretization (e.g. Method-of-Lines), hybrid DAE–PDE, coupling with control / optimization / ML is straightforward. Good for custom, advanced, hybrid modeling.                                                                                                             |
| Solver Integration                    | Modelica tools compile to DAE/ODE and use bundled or internal solvers; solver call is automatic after compilation.                                                                                                                  | MTK lowers symbolic models to Julia-native systems and leverages the SciML solver ecosystem (ODE, DAE, SDE, PDE, hybrid, event, sensitivity, optimization, etc.).                                                                                                                                                    |
| Symbolic Preprocessing / Transforms   | Compiler handles transformations (index-reduction, simplification, variable elimination, algebraic-loop resolution) internally; user has limited control or visibility.                                                             | Fully transparent/exposed symbolic backend: model represented as symbolic equation graph (via Symbolics.jl); user can inspect, transform, simplify, reorder equations, perform index reduction, sparsification, generate efficient code (Jacobians/Hessians), parallelize — and define custom transformation passes. |
| Extensibility / Research-Friendliness | Extending means writing new model libraries or components; modifying compiler transformations is difficult and often requires deep expertise or forking — the “two-language problem” remains (modeling DSL vs external scripting).  | High extensibility: modeling, simulation, optimization, data analysis, ML — all in Julia. The modular transformation and compilation stack is open, enabling custom passes, surrogate models, tight integration with optimization or ML workflows — very research-friendly.                                          |
| Tooling / GUI / Visual Modeling       | Mature commercial and open-source tools with graphical editors, drag-and-drop component assembly, visual connectors, parameter dialogs — widely used in industry.                                                                   | Mostly code-based modeling (Julia code); limited GUI or drag-and-drop support — modeling tends to be textual/symbolic, better suited for scripting, version control, automated workflows than visual modeling.                                                                                                       |
| Best Suited For                       | Standard multi-domain physical modeling tasks using established libraries.                                                                                                                                                          | Research, custom/hybrid modeling, advanced                                                                                                                                                                                                                                                                           |

## Discussion

Modelica provides a mature, well-tested ecosystem with built-in connectors, standard libraries, and visual modeling tools — making it a robust choice for conventional engineering tasks, multi-domain component composition, and industrial workflows. Its top-down, DSL-based style with ready components lowers entry barriers and speeds up development when standard building blocks suffice.

On the other hand, ModelingToolkit.jl (MTK) offers maximal flexibility: symbolic-numeric modeling embedded in a full-featured language (Julia), open and extensible transformation pipelines, seamless coupling with solvers, optimization, machine learning, and user-defined components. For research-oriented applications — especially those requiring custom PDE/DAE models, hybrid dynamics, control/optimization integration, or custom connectors — MTK’s flexibility and transparency represent significant advantages. The trade-off is that modeling is code-centric, and GUI/drag-and-drop convenience is limited compared to Modelica.

Given typical research and development workflows — custom hydropower simulation, novel component development, PDE/DAE coupling, control and optimization in one unified framework — MTK is especially compelling. For standard engineering tasks where mature libraries suffice and graphical modeling is preferred, Modelica remains a powerful, convenient option.

## 1 Acknowledgement

I acknowledge the use of ChatGPT (OpenAI, <https://chat.openai.com/>) to generate a draft of portions of this report — in particular, the initial comparison text between Modelica and ModelingToolkit.jl. I refined, edited, and fact-checked the content to ensure technical accuracy and coherence, and all final formulations are my own. Acknowledgement

## References

- [ModelingToolkit.jl](#) — official documentation and overview
- “Modeling complexity with Symbolics.jl and ModelingToolkit.jl” — blog post on acausal symbolic modeling
- [ModelingToolkit.jl](#) — PDESystem interface for PDE modeling
- [SciML / DifferentialEquations.jl](#) — solver suite for Julia
- “ModelingToolkit: A Composable Graph Transformation System for Equation-Based Modeling” — core MTK paper